

Web Technologies & Security

Practical 5

Task 2

security test that you do must be supported by proof of concept and a screenshot or copy of your code used to implement the security feature in mention. Show the problem identified and how you have mitigated it. In this task you are expected to do the following:

1. What security measures/features have you implemented to safeguard your website against:

- Loss of availability through SQL injection
- Loss of confidentiality through XSS
- Loss of integrity through user session hijacking
- Insecure Direct Object References and File traversal

```
<?php

if($_SERVER["REQUEST_METHOD"] == "POST"){
    $rating= validate($_POST["rating"]);
    $comment= validate($_POST["review"]);

    try{
        require_once "connection.php";

        $mysql ="INSERT INTO review (ReviewComment, Rating) VALUES (?,?)";
        $sql = $conn->prepare($mysql);
        $sql->execute([ $comment, $rating]);
        $conn = null;
        $sql = null;
        header("Location: review.php");
        die();
    }catch(PDOException $e){
        die("Error found: ". $e->getMessage());
    }
}

5 references
function validate($input){
    $input=trim($input);
    $input=stripslashes($input);
    $input=htmlspecialchars($input);
    return $input;
}
```

Validates all values entered into the review form.

```

<?php

if($_SERVER["REQUEST_METHOD"] == "POST"){
    $perp= validate($_POST["perp"]);
    $misconduct=validate($_POST["misconduct"]);
    $description= validate($_POST["description"]);

    try{
        require_once "connection.php";

        $query ="INSERT INTO report (MisconductType, MisconductDescription, Reported
        $stmt = $conn->prepare($query);
        $stmt->bind_param("sss", $misconduct, $perp, $description);
        $stmt->execute();
        $stmt->close();
        $conn->close();
        header("Location: report.php");
        die();
    }catch(Exception $e){
        die("Error found: ". $e->getMessage());
    }
}

5 references
function validate($input){
    $input=trim($input);
    $input=stripslashes($input);
    $input=htmlspecialchars($input);
    return $input;
}

```

Similarly the code for the report form, checking all the security requirements of number one except for 1,d). While other files aren't in view when uploading them, you can still choose other files and they will be seen as images and stored in the database. However no malicious programming can infiltrate the website.

2. How have you implemented your authentication system?

a.

```

92     $status = "Username or email already exists.";
93 } else {
94     // Insert new user into database after hashing password
95     $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
96     $insertQuery = $conn->prepare("INSERT INTO user (username, email, password, permission) VALUES (?, ?, ?, ?)");
97     $insertQuery->bind_param("ssss", $username, $email, $hashedPassword, $usertype); // Default permission set to 'user'

```

```

36
37     // 2. Verify the password
38     if (password_verify($password_input, $user['password'])) {
39         // 3. Create Session variables

```

Password hashing before storage and password verify when fetching it from storage.

```

16
17
18 <?php
19 session_start();
20 require "connection.php";
21
22 $error_message = "";
23
24 if ($_SERVER["REQUEST_METHOD"] == "POST") {
25     $username_input = $_POST['username'];
26     $password_input = $_POST['password'];
27
28     // 1. Prepare a statement to fetch the user by username or email
29     $stmt = $conn->prepare("SELECT UserID, username, password FROM user WHERE username = ? OR email = ?");
30     $stmt->bind_param("ss", $username_input, $username_input);
31     $stmt->execute();
32     $result = $stmt->get_result();
33
34     if ($result->num_rows === 1) {
35         $user = $result->fetch_assoc();
36
37         // 2. Verify the password
38         if (password_verify($password_input, $user['password'])) {
39             // 3. Create Session variables
40             $_SESSION['user_id'] = $user['UserID'];
41             $_SESSION['username'] = $user['username'];
42             $_SESSION['logged_in'] = true;
43
44             // Redirect to home page
45             header("Location: index.php");
46             exit();
47         } else {
48             $error_message = "Invalid password.";
49         }
50     } else {
51         $error_message = "No account found with that username or email.";
52     }

```

Using sessions to verify that a user is currently/ already signed

```

72
73 <?php
74 if (!isset($_SESSION['user_id'])) {
75     echo "<p>Please log in to see your listings.</p>";
76 } else {
77     $userId = $_SESSION['user_id'];
78
79     $sql = "SELECT all_listing.* FROM all_listing
80         JOIN user_listing ON all_listing.listingID = user_listing.listingID
81         WHERE user_listing.UserID = ?";
82
83     $stmt = $conn->prepare($sql);
84     $stmt->bind_param("i", $userId);
85     $stmt->execute();
86     $result = $stmt->get_result();
87

```

Using sessions to display their account listings and to enable them to change password

```

logout.php
1 <?php
2 session_start();
3 session_unset(); // Clear all session variables
4 session_destroy(); // Kill the session
5 header("Location: index.php"); // Send back to home
6 exit();
7 ?>

```

Using sessions to log out a user from their account

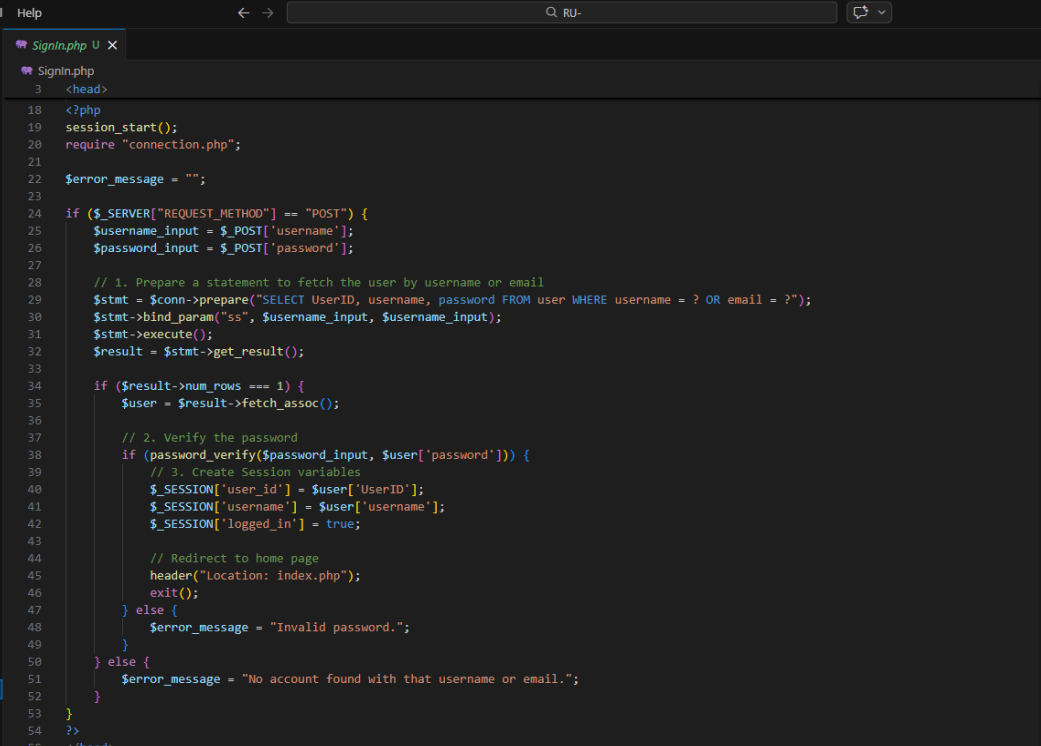
```

16
17 //prepared statements to prevent sql injection
18 $stmt = $conn->prepare("SELECT * FROM user WHERE username=? OR email=?");
19 $stmt->bind_param("ss", $username, $email);
20 $stmt->execute();
21 $result = $stmt->get_result(); //get result object
22

```

Prepared statements to prevent sql injection

b. Show your code and explain how it works to authenticate users



```

Help
Signin.php U x
3 <head>
18 <?php
19 session_start();
20 require "connection.php";
21
22 $error_message = "";
23
24 if ($_SERVER["REQUEST_METHOD"] == "POST") {
25     $username_input = $_POST['username'];
26     $password_input = $_POST['password'];
27
28     // 1. Prepare a statement to fetch the user by username or email
29     $stmt = $conn->prepare("SELECT UserID, username, password FROM user WHERE username = ? OR email = ?");
30     $stmt->bind_param("ss", $username_input, $username_input);
31     $stmt->execute();
32     $result = $stmt->get_result();
33
34     if ($result->num_rows == 1) {
35         $user = $result->fetch_assoc();
36
37         // 2. Verify the password
38         if (password_verify($password_input, $user['password'])) {
39             // 3. Create Session variables
40             $_SESSION['user_id'] = $user['UserID'];
41             $_SESSION['username'] = $user['username'];
42             $_SESSION['logged_in'] = true;
43
44             // Redirect to home page
45             header("Location: index.php");
46             exit();
47         } else {
48             $error_message = "Invalid password.";
49         }
50     } else {
51         $error_message = "No account found with that username or email.";
52     }
53 }
54 ?>
55 </head>

```

Users are authenticated through username and password. After they enter their details, prepared statements prevent SQL injection by sending placeholders '?' to the database and binding variables to their data types. If the username and password match what is in the database, session variables are set.

3. Write your own PHP script that reads the log file and detect login attempts

```
<?php
session_start();

1 reference
function readLoginLog() {
    $logFile = 'login_attempts.log';
    $attempts = [];

    if (!file_exists($logFile)){
        return ['error' => 'No login attempts logged yet'];
    }

    //Read the file line by line
    $lines = file($logFile, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);

    foreach($lines as $line){
        if (preg_match('/\[([.?:\s]) IP: ([.?:\s]) \| User: ([.?:\s]) \| Status: ([.?:\s]) \|/', $line, $matches)) {
            $attempts[] = [
                'time' => $matches[1],
                'ip' => $matches[2],
                'user' => $matches[3],
                'status' => $matches[4]
            ];
        }
    }

    //return in reverse so new one is first
    return array_reverse($attempts);
}
```

A piece of the logfile which ensures no brute force attacks can pass or go through, I will add that this is highly theoretical and we weren't able to test this out for fear of having our website damaged permanently.

4. Apart from the security features mentioned in option 1 of Task 1, what other security features, overall, have you implemented on your system's:
- a. front end [5 Marks]

```
<?php if ($error_message): ?>
    <p style="color: red; text-align: center;"><?php echo $error_message; ?></p>
<?php endif; ?>

<section>
    <div class="signin-username">
        <input type="text" placeholder="Username or Email" id="username" name="username" required>
    </div>

    <br>

    <div class="signin-password">
        <input type="password" placeholder="Password" id="password" name="password" style="position: relative;" required>
        <div class=""></div><i class="fa fa-eye hide"></i>
        <i class="fa fa-eye-slash hide" ></i>
    </div>
</section>

</body>
</html>

<form autocomplete="off" id="form" method = "POST" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>" >
    <h2 style="color: rgb(33, 116, 103);>Create Account</h2>

    <p id="Err-Messages"><?php echo htmlspecialchars($status); ?></p> <!-- status msg from PHP -->

    <div class="createacc-firstname">
        <input type="text" placeholder="Username" id="username" name="Username" required>
    </div>
    <br>

    <div class="email">
        <input type="email" placeholder="Email" id="email" name="Email" required>
    </div>
    <br>

    <div class="createacc-password">
        <input type="Password" placeholder="Password" id="Password" name="Password" minlength="8" required>
    </div>
    <br>

    <div class="createacc-cpassword">
        <input type="Password" placeholder="Confirm password" id="Cpassword" name="CPassword" minlength="8" required>
    </div>
    <br>

    <button type="submit" class="btn">Create Account</button>
    <p class="already" >Already have an Account<a href="SignIn.php"> Sign In</a></p>
</form>
```

As can be seen from the above 2 screenshots of our code, we implemented some front-end security features in our html that requires some of the fields for our sign-in and create account pages. We also added a minimum length requirement of 8 when creating a password, just so that it wouldn't be too easy for anyone to guess when trying to access a user's account.

```

<?php if (isset($_SESSION['user_id'])): ?>
    <li><a href="logout.php">Logout</a></li>
    <li><a href="listing.php">Create Listing</a></li>
<?php else: ?>
    <li><a href="Create_Account.php">Create Account</a></li>
    <li><a href="SignIn.php">Sign in</a></li>
<?php endif; ?>
<li><a href="CampusMap.php">Map</a></li>
<li><a href="about.php">About Us</a></li>

```

```

61 | | | | </div>
62 | | | |
63 | <div class="nav-icons">
64 |     <?php if (isset($_SESSION['user_id'])): ?>
65 |         <a href="MyAccount.php"></a>
66 |     <?php else: ?>
67 |         <a href="SignIn.php"></a>
68 |     <?php endif; ?>
69 |
70 | </div>
71 |

```

There is also the principle of least privilege for guests, for users, and for admins

b. back end [5 Marks]

```

// Validate input
if (empty($username) || empty($email) || empty($password) || empty($confirmPassword)) {
    $status = "All fields are required.";
} elseif ($password != $confirmPassword) {
    $status = "Passwords do not match.";
} else {
    if ($result->num_rows > 0) {
        $status = "Username or email already exists.";
    } else {
        // Insert new user into database after hashing password
        $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
        $insertQuery = $conn->prepare("INSERT INTO user (username, email, password, permission) VALUES (?, ?, ?, ?)");
        $insertQuery->bind_param("ssss", $username, $email, $hashedPassword, $usertype); // Default permission set to 'user'
        if ($insertQuery->execute()) {
            $status = "Account created successfully!";
            echo
            header("Location: SignIn.php"); // Redirect to sign-in page after successful account creation
        } else {
            $status = "Error: " . $conn->error;
        }
    }
}

```

```

changePassword.php
# ($_SERVER["REQUEST_METHOD"] == "POST") {

    if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] != true || !isset($_SESSION['user_id'])) {
        header('Location: SignIn.php');
        exit;
    }

    $oldPassword = $_POST["Curpassword"];
    $newPassword = $_POST["Nwpassword"];
    $confirmPassword = $_POST["Cfpassword"];
    $userId = $_SESSION['user_id']; //from sign in session

    if (empty($oldPassword) || empty($newPassword) || empty($confirmPassword)) {
        $status = "All fields are required.";
    } elseif ($newPassword != $confirmPassword) {
        $status = "New passwords do not match.";
    } else {

        $stmt = $conn->prepare("SELECT password FROM user WHERE UserID = ?");
        $stmt->bind_param("i", $userId);
        $stmt->execute();
        $result = $stmt->get_result();
        $user = $result->fetch_assoc();

        if ($user && password_verify($oldPassword, $user['password'])) {
            $hashedPassword = password_hash($newPassword, PASSWORD_DEFAULT);
            $SQLQuery = $conn->prepare("UPDATE user SET password = ? WHERE UserID = ?"); //new password update query
            $SQLQuery->bind_param("si", $hashedPassword, $userId);

            if ($SQLQuery->execute()) {
                //display
                echo '<script>
                    alert("Password changed successfully!");
                    window.location.href = "index.php";
                </script>';
                exit;
            } else {
                $status = "Error: " . $conn->error;
            }
        } else {
            $status = "Current password is incorrect.";
        }
    }
}

```

```

    if ($result->num_rows > 0) {
        $status = "Username or email already exists.";
    } else {
        // Insert new user into database after hashing password
        $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
        $insertQuery = $conn->prepare("INSERT INTO user (username, email, password, permission) VALUES (?, ?, ?, ?)");
        $insertQuery->bind_param("ssss", $username, $email, $hashedPassword, $usertype); // Default permission set to 'user'
        if ($insertQuery->execute()) {
            $status = "Account created successfully!";
            echo
            header('Location: SignIn.php'); // Redirect to sign-in page after successful account creation
        } else {
            $status = "Error: " . $conn->error;
        }
    }
}

```

As shown in the above screenshots we implemented these backend security features too. In the first screenshot we see how we integrated a validation system that checks if the user's password is correct by referring to the password stored in the database. As shown in the second screenshot we made it so that when changing your password, you first have to type in your previous password, and then after typing your new password you are asked to confirm it. We also made it so that whenever you change your password, or create an account for the first time, the password is hashed before being put into the database.

c. Show your code and work for this
Shown in the screenshots provided.

5. Additional points will be allocated for creativity and use of functions or attributes not directly stated. In essence, if you follow all of this your best mark will be 40/50. 10% has been left for grabs for initiative (read additional use of security features not stated). Make sure you comment this clearly in your file so that your tutors can see it and award you the marks. [10 Marks]